

1. gRPC E2E テスト手順書

Go 製の図書照会サービスに対して、runnora で Unary RPC と Server streaming RPC を実行する。API 定義は `proto/library.proto`、runbook は `runbooks/` に置く。

`grpc-test/` で `./scripts/run.ps1` を実行してテストし、`./scripts/build-docs.ps1` でこの手順書を生成する。前処理用 DB、oapi2wire、モックサーバは使用しない。

2. Unary RPC

GetBook で B0003 を照会し、gRPC ステータスと蔵書の内容を確認する。

表 2-1 テストシナリオ

シナリオ	Unary RPC で蔵書を 1 冊取得する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	B0003 のタイトル、ジャンル、在庫数を確認する Unary sample.library.v1.LibraryService/GetBook	表 2-3	期待値 : [gRPC 0] 表 2-4	

表 2-2 gRPC 呼び出し情報 (Unary RPC で蔵書を 1 冊取得する)

手順	Runner	Address	Service/Method	RPC type	Metadata	Timeout
1	greq	127.0.0.1:19090	sample.library.v1.LibraryService/ GetBook	Unary		

表 2-3 gRPC 送信メッセージ (Unary RPC で蔵書を 1 冊取得する / 手順 1)

JSON path	値
\$.book_id	B0003

表 2-4 期待値・検証条件 (Unary RPC で蔵書を 1 冊取得する / 手順 1)

区分	出典	内容
検証式	unary.yml:15	current.res.status == 0 && current.res.message.book.book_id == "B0003" && current.res.message.book.title == "Go 言語による Web アプリ開発" && current.res.message.book.genre == "TECH" && current.res.message.book.available_copies == 2
ステータス	検証式から抽出	gRPC 0

3. Server streaming RPC

Unary `GetBook` の応答からジャンルを `bind` で保存し、`ListBooks` の入力に使う。受信したメッセージ配列の件数、順序、内容を検証する。

表 3-1 テストシナリオ

シナリオ	Unary の応答を使って Server streaming RPC の蔵書一覧を検証する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	検索条件に使う蔵書のジャンルを Unary RPC で取得する Unary <code>sample.library.v1.LibraryService/GetBook</code>	表 3-3	期待値 : [gRPC 0] 表 3-5	
2	応答のジャンルを後続の streaming RPC へ渡す 変数を設定			
3	TECH の蔵書が ID 順に 2 件流れることを確認する Server streaming <code>sample.library.v1.LibraryService/ListBooks</code>	表 3-4	期待値 : [gRPC 0] 表 3-6	

表 3-2 gRPC 呼び出し情報 (Unary の応答を使って Server streaming RPC の蔵書一覧を検証する)

手順	Runner	Address	Service/Method	RPC type	Metadata	Timeout
1	greq	127.0.0.1:19090	<code>sample.library.v1.LibraryService/GetBook</code>	Unary		
3	greq	127.0.0.1:19090	<code>sample.library.v1.LibraryService/ListBooks</code>	Server streaming		

表 3-3 gRPC 送信メッセージ (Unary の応答を使って Server streaming RPC の蔵書一覧を検証する／手順 1)

JSON path	値
<code>\$.book_id</code>	B0003

表 3-4 gRPC 送信メッセージ (Unary の応答を使って Server streaming RPC の蔵書一覧を検証する／手順 3)

JSON path	値
<code>\$.genre</code>	<code>{{ genre }}</code>

表 3-5 期待値・検証条件（Unary の応答を使って Server streaming RPC の蔵書一覧を検証する／手順 1）

区分	出典	内容
検証式	server-streaming.yml:15	current.res.status == 0 && current.res.message.book.genre == "TECH"
ステータス	検証式から抽出	gRPC 0

表 3-6 期待値・検証条件（Unary の応答を使って Server streaming RPC の蔵書一覧を検証する／手順 3）

区分	出典	内容
検証式	server-streaming.yml:28	current.res.status == 0 && len(current.res.messages) == 2 && current.res.messages[0].book_id == "B0003" && current.res.messages[0].genre == genre && current.res.messages[1].book_id == "B0004" && current.res.messages[1].genre == genre
ステータス	検証式から抽出	gRPC 0